1 Mi juego en pocas palabras

Marihuanero explicado para mi exposición

Guía de estudio del proyecto y sus scripts

Esta guía explica la versión guardada del proyecto Marihuanero en Unity 6.6. Está escrita para entender el juego y poder enseñarlo con tus propias palabras. No hace falta memorizar cada línea de código.

Qué es y cómo se gana

Es un juego de plataformas en 2D con una selva como escenario. El personaje recorre el nivel, salta entre plataformas y recoge diez hojas. Después debe tocar al Patrón que está al final. Tener las diez hojas no termina la partida por sí solo: todavía hay que llegar a la meta.

Los pinchos y las caídas son los peligros. Si toco pinchos o caigo demasiado abajo, aparece la derrota. Si llego a la meta con las diez hojas, aparece la victoria. En ambos casos puedo volver a empezar.

Acción	Cómo la hago
Caminar	A y D o flechas izquierda y derecha
Saltar	Barra espaciadora mientras estoy en el suelo
Empezar	Botón JUGAR en el menú
Volver a intentar	Botón del panel final

Lo que debo saber para mañana

Primero entiende el recorrido completo: menú, movimiento, hojas, peligro y final. Después aprende qué tarea tiene cada script. Al final de esta guía hay un discurso corto y preguntas para practicar. Puedes tener Unity abierto para señalar las cosas mientras explicas. Las páginas 18 a 25 incluyen código real de los ocho scripts y frases para explicarlo en la defensa.

La carpeta del proyecto es C:\Users\soyel\Marihuanero. Los ocho scripts del juego están en Assets/Scripts. En esta revisión solo se añadieron comentarios: las instrucciones que hacen funcionar el juego siguen iguales.

2 Cómo está organizado el proyecto

Las ventanas que uso en Unity

Project muestra las carpetas y archivos del proyecto. Hierarchy muestra los objetos de la escena que tengo abierta. Inspector muestra las casillas y piezas del objeto seleccionado. Scene sirve para acomodar las cosas; Game muestra lo que ve la cámara del jugador.

Por eso, cuando abro MenuPrincipal ya no veo las plataformas en Hierarchy. No significa que se borraron: están dentro de juegofuncional. Para verlas otra vez abro esa escena desde la carpeta Scenes.

Carpeta dentro de Assets	Para qué sirve
Scenes	Guarda MenuPrincipal y juegofuncional
Scripts	Guarda las instrucciones escritas en C#
Imagenes	Fondos personajes suelo hojas pinchos y retratos
Prefabs	Guarda moldes de objetos que puedo copiar
Sonidos	Efectos y música de fondo
Materiales	Apariencia y efecto de la atmósfera
Settings y TextMeshPro	Ajustes de Unity y recursos para mostrar textos

Cómo se empezó

Se creó un proyecto 2D. Primero se probaron las reglas con figuras sencillas: un personaje, piso, plataformas y objetos de colores. Cuando esas reglas funcionaron, se cambiaron los dibujos por imágenes de la selva. Así no hubo que volver a programar todo para cambiar la apariencia.

Las dos escenas

MenuPrincipal presenta el juego y sus botones. juegofuncional contiene el nivel. En Build Profiles, las dos escenas están en la lista y el menú va primero. Esto permite que el botón JUGAR abra el nivel por su nombre.

Un objeto puede tener varias piezas. Por ejemplo, el jugador tiene una pieza para la física, otra para detectar choques y scripts para moverse y animarse. Unity permite unirlas en el Inspector.

3 Cómo se construyó el recorrido

El dibujo y la parte que se puede tocar

El Sprite Renderer enseña la imagen. El Box Collider 2D es una caja invisible que marca dónde se puede chocar o tocar. El personaje no camina sobre los colores de la imagen: se apoya sobre esa caja. Por eso hay que ajustar ambos para que los pies se vean sobre el suelo.

El suelo usa tierra con césped arriba y profundidad hacia abajo. Las plataformas tienen una imagen de tierra flotante. Cambiar una imagen no reemplaza automáticamente el tamaño de la caja: eso se revisa por separado.

El nivel que tengo ahora

El recorrido tiene doce plataformas, diez hojas y doce grupos de pinchos. Hay dos huecos, aproximadamente entre X 24 y 29 y entre X 58 y 63. El Patrón está cerca de X 89. Arbol1 y Arbol2 sirven como límites sólidos para no salir por los extremos.

X significa izquierda o derecha. Y significa altura. Z se usa para la profundidad. Las plataformas están a distintas alturas para que el jugador tenga que calcular sus saltos. El suelo tiene su parte superior cerca de Y menos 2,5.

Para qué hicimos los prefabs

Un prefab es un molde: guarda el dibujo, el tamaño, la caja y la etiqueta de un objeto. El administrador usa el molde de hoja y el de pinchos para crear copias al empezar. Las posiciones están guardadas en listas; así no necesitamos programar cada copia por separado.

La etiqueta o Tag ayuda a reconocer qué tocamos. Hoja significa recoger; Pinchos significa perder; Meta significa comprobar si ya ganamos. La capa Suelo ayuda a reconocer dónde se puede apoyar el personaje.

Cómo lo muestro en la defensa

Selecciono una plataforma y señalo su Sprite Renderer y su Box Collider 2D. Luego abro un prefab y explico que es un molde. Al entrar en Play aparecen las copias de hojas y pinchos que crea el programa.

4 ControlJugador y el movimiento

Este archivo va en el objeto Jugador. Su tarea es leer el teclado, mover el cuerpo, permitir el salto y avisar cuando tocamos algo importante.

Primero busca lo que necesita

Awake guarda el Rigidbody2D y el BoxCollider2D del personaje. El primero permite moverlo con física y gravedad; el segundo marca su caja. GetComponent significa buscar esa pieza en el mismo objeto.

Después escucha el teclado

Update revisa las teclas en cada imagen del juego. Al comenzar deja dirección en cero. Si pulso izquierda resta uno y si pulso derecha suma uno. Si pulso ambas, se cancelan. Si no hay teclado conectado, sale de ese bloque sin seguir leyendo teclas.

Espacio deja un aviso de salto pendiente. Es como decir “quiero saltar”; el bloque de física decide si realmente se puede. Si la partida ya terminó, no sigue aceptando movimiento. También revisa si el jugador cayó por debajo de Y menos 7 para pedir la derrota.

La parte que mueve de verdad

```
Vector2 velocidad = new Vector2(
    direccion * rapidez,
    cuerpo.linearVelocity.y
);
```

El primer número mueve a los lados. El segundo conserva la subida o caída que ya tenía el personaje. Si pusiera cero en el segundo número todo el tiempo, estorbaría el salto y la gravedad.

Cómo sabe si puede saltar

FixedUpdate hace una pequeña revisión debajo de los pies con OverlapBox. Solo busca objetos de la capa Suelo. Si hay un apoyo y el salto está pendiente, pone una velocidad hacia arriba y pide el sonido de salto. Después borra el aviso para no saltar muchas veces con una sola pulsación.

En el Inspector actual, la rapidez es 5, la fuerza de salto es 10 y la gravedad es 2. Son las casillas guardadas en Unity. El número inicial escrito en el archivo puede ser distinto: cambiar ese número no siempre cambia una casilla que ya estaba guardada.

5 ControlJugador y lo que toca

La entrada de los contactos

OnTriggerEnter2D se ejecuta cuando el personaje entra en una zona marcada como Is Trigger. Esa zona avisa del contacto sin convertirse en una pared. El suelo normal usa una caja sólida; las hojas, los pinchos y la meta usan zonas que avisan.

```
if (otro.CompareTag("Hoja"))
```

CompareTag pregunta si el objeto tiene esa etiqueta. El nombre debe coincidir exactamente, incluyendo mayúsculas. No se decide por el color verde o por la forma del dibujo.

Cuando toca una hoja

Primero se desactiva la caja de la hoja para que el mismo contacto no la cuente dos veces. Después se llama a RecogerHoja del administrador. Finalmente se elimina esa copia del nivel con Destroy. El molde guardado en Prefabs no se borra.

Cuando toca pinchos

El jugador llama a Morir del administrador. El administrador controla el sonido, el panel y la pausa del nivel. El personaje no necesita repetir esas instrucciones para cada grupo de pinchos.

Cuando toca la meta

Llama a IntentarGanar. Ese nombre tiene sentido porque llegar al final no basta: el administrador todavía revisa cuántas hojas tenemos. Si faltan hojas, avisa y deja continuar. Si tenemos diez, muestra la victoria.

Un ejemplo completo para explicar

“Yo pulso D y el personaje camina. Cuando su caja toca la zona de una hoja, el script reconoce la etiqueta Hoja. Le avisa al administrador para sumar una y quitar esa copia. La cuenta cambia en la pantalla y suena el efecto. Cada parte tiene su propia tarea”.

Por qué se separó así

ControlJugador se ocupa del personaje. ControlNivel se ocupa de las reglas de la partida. Así, si quiero cambiar el mensaje de victoria, voy al administrador y no tengo que revisar toda la lectura del teclado.

6 ControlNivel y el comienzo de la partida

ControlNivel está en AdministradorNivel. Es como el encargado del recorrido: guarda la cuenta, crea objetos y decide cuándo termina la partida.

Las casillas del principio

textoHojas y textoAviso son textos de pantalla. prefabHoja y prefabPinchos son los moldes. panelDerrota y panelVictoria son los letreros finales. sonidos apunta al encargado del audio. Estas conexiones se hacen arrastrando los objetos correctos al Inspector.

hojasRecogidas guarda cuántas llevamos. totalHojas vale diez. Las dos marcas derrotaMostrada y victoriaMostrada recuerdan si ya terminó la partida. PartidaTerminada responde sí cuando cualquiera de las dos está encendida.

Lo que hace Start

Al entrar al nivel devuelve el tiempo a velocidad normal, deja el contador en cero, limpia el aviso y esconde los dos paneles finales. Después crea las hojas, crea los pinchos y escribe el contador inicial. Así cada intento comienza de la misma manera.

Las listas de lugares

```
foreach (Vector2 posicion in posicionesHojas)
    Instantiate(prefabHoja,
        new Vector3(posicion.x, posicion.y, 0f),
        Quaternion.identity);
```

foreach significa “por cada lugar de la lista”. Instantiate significa “crea una copia del molde”. Vector3 reúne X, Y y Z. Quaternion.identity deja la copia sin un giro añadido. No tienes que memorizar la palabra larga: aquí solo indica que la hoja aparece derecha.

CrearHojas usa una lista con X y Y. CrearPinchos usa una lista de X y les da una misma altura de Y menos 2,2. Las listas se pueden cambiar desde el Inspector.

La cuenta que veo

RecogerHoja suma una, pide su sonido y llama a ActualizarContador. Este último escribe algo como Hojas: 3 / 10 en el texto conectado. La pantalla muestra la cuenta que guarda el programa; no cuenta dibujos por su cuenta.

7 ControlNivel y los dos finales

Cómo funciona la victoria

IntentarGanar primero revisa si la partida ya terminó. Si todavía seguimos y faltan hojas, escribe un aviso. Si ya tenemos las diez, marca victoria, reproduce el sonido, limpia el aviso, muestra PanelVictoria y detiene el tiempo del juego.

Cómo funciona la derrota

Morir también sale enseguida si la partida había terminado. Si no, marca derrota y pide el sonido de muerte antes de mostrar el panel y parar el tiempo. Esto evita que el mismo peligro vuelva a pedir muchas derrotas.

```
Time.timeScale = 0f;
```

Esta línea deja quieta la parte del juego que usa ese tiempo: movimiento y física, por ejemplo. El audio puede seguir sonando y los botones de la pantalla final siguen funcionando. Por eso se puede escuchar el meme y luego pulsar Reintentar.

Cómo funciona volver a jugar

```
Time.timeScale = 1f;
SceneManager.LoadScene(
    SceneManager.GetActiveScene().name
);
```

ReiniciarNivel vuelve a poner el tiempo normal y carga otra vez la escena actual. Al cargarse se ejecuta de nuevo Start. Por eso las hojas reaparecen y el contador vuelve a cero sin tener que devolver cada hoja a mano.

Cómo conecto el botón

Selecciono el botón del panel. En Button busco On Click, pulso más y arrastro AdministradorNivel al espacio del objeto. En la lista elijo ControlNivel y luego ReiniciarNivel. Debo arrastrar el objeto que tiene el script, no el archivo del script ni el propio botón.

Un error que aprendimos a reconocer

NullReferenceException suele indicar que una casilla necesita algo y está vacía. Por ejemplo, si textoHojas no apunta al texto de la pantalla, el programa no sabe dónde escribir. Reviso el mensaje de Console y la casilla correspondiente del Inspector.

8 AnimarJugador y los dibujos del personaje

El personaje tiene un cuerpo que se mueve y una imagen que se ve. La imagen está en el hijo ImagenJugador. Esta separación permite cambiar el dibujo sin cambiar las reglas de movimiento.

Cómo se prepararon las imágenes

PersonajeRelajado.png reúne doce poses en una sola imagen. Unity las separa en cuadritos. Cuatro forman Quieto, cuatro Caminar y cuatro Saltar. Los archivos de animación indican qué dibujos enseñar y en qué orden. Un PNG por sí solo no guarda una animación que Unity reproduzca automáticamente.

Jugador.controller tiene los tres estados. En este proyecto el script elige el estado que corresponde; no depende de una red de flechas para decidirlo. El dibujo tampoco arrastra el cuerpo: Apply Root Motion está desactivado.

Qué hace cada bloque

Awake guarda el cuerpo, la caja y ControlJugador. Update revisa si la partida está en pausa, lee la velocidad y comprueba el apoyo debajo de los pies. Después pasa esos datos a MostrarMovimiento.

MostrarMovimiento mira si estamos en el aire, caminando o quietos. También voltea el dibujo cuando caminamos a la izquierda. Recuerda el estado anterior para no reiniciar la misma animación a cada momento.

MostrarPoseSalto enseña una parte concreta de la animación. Cuando subimos muestra la pose de subida; cuando caemos, la de caída. Al tocar el piso mantiene brevemente una pose de aterrizaje, aproximadamente una décima de segundo, antes de volver a caminar o estar quieto.

Lo que digo si me preguntan

“La animación mira lo que el cuerpo ya está haciendo. No es la animación la que calcula el salto. Primero se mueve el cuerpo y después se elige el dibujo que combina con ese movimiento”.

Cómo lo enseño

Abro la carpeta de personajes y las animaciones. Luego doy Play y pruebo caminar, detenerme, saltar y cambiar de dirección. Señalo el objeto Jugador y su hijo ImagenJugador para explicar la separación.

9 La cámara y el tamaño del fondo

CamaraSigue

Este script está en Main Camera y tiene conectado al Jugador. OnEnable prepara el seguimiento desde cero. LateUpdate trabaja después del movimiento del personaje para colocar la cámara usando su posición más reciente.

En el primer momento coloca el encuadre dos unidades delante del personaje. Después acompaña el cambio horizontal que hizo el jugador y suaviza la corrección que falte. Conserva la altura y la profundidad: no sube y baja con cada salto.

Antes había una sensación de empujón hacia atrás. Se corrigió el seguimiento para acompañar el movimiento horizontal mostrado en pantalla. No se añadió una fuerza de retroceso al personaje. Además, la física del jugador usa Interpolate para que se vea más suave y detección Continuous para los contactos.

El acercamiento actual

La cámara es Orthographic, que muestra una vista plana apropiada para 2D. Su Size actual es 4,2. Ese número representa la mitad de la altura visible. Al bajarlo se ve menos escenario y el personaje se ve más grande. No hace falta agrandar al personaje para acercar la vista.

AjustarFondo

OnEnable encuentra la cámara y la imagen. LateUpdate revisa el tamaño visible. AjustarTamano calcula cuánto debe crecer el fondo para cubrir la pantalla, manteniendo sus proporciones. También tiene en cuenta el tamaño del objeto padre.

Elige el aumento que alcance a cubrir tanto el ancho como el alto. Por eso puede quedar una parte de la imagen fuera de la pantalla: es preferible a mostrar un borde vacío. Solo cambia la escala cuando hace falta.

Dónde se usa

AjustarFondo sirve para el fondo del nivel y para FondoPortada del menú. CamaraSigue pertenece a la cámara del nivel. La cámara del menú muestra su presentación y no persigue al jugador.

10 La selva y su ambiente animado

Lo que realmente se animó

El nivel combina una imagen de selva con una capa transparente de niebla, luces y puntitos. No se puso un GIF de diez segundos. El efecto se dibuja con un material y se repite suavemente para dar vida al fondo.

AnimarSelva

OnEnable busca la capa que dibuja el efecto y prepara sus valores. LateUpdate va avanzando un reloj mientras jugamos. El ciclo está pensado para repetirse cada diez segundos. Cuando termina la partida, el tiempo del juego se detiene y el ambiente deja de avanzar.

ActualizarCapa hace que esa capa cubra la cámara. Usa un suelo de referencia para saber dónde está el piso y colocar la niebla a una altura coherente. Así la niebla no queda en cualquier lugar cuando cambia el encuadre.

El script manda al material la fase del ciclo, la forma de la pantalla, la cantidad de niebla, la intensidad de rayos, el brillo de partículas y sus colores. MaterialPropertyBlock es una forma de cambiar esos valores sin fabricar un material nuevo a cada momento. OnDisable limpia los valores que aplicó a la capa.

El archivo AtmosferaSelva shader

Un shader es una receta para pintar. Este dibuja una capa transparente. Sus pequeñas funciones Azar, Ruido y Nube crean variaciones para que la niebla no sea un rectángulo de color plano. La parte final combina esa niebla, haces de luz y puntos que se mueven en un ciclo.

No cambia dónde se puede caminar y no empuja al personaje. Solo cambia lo que se ve. Tampoco es una película: el efecto se calcula mientras el juego está funcionando.

La diferencia con el menú

El menú actual usa la portada que proporcionaste. La niebla de selva pertenece al nivel; no es el fondo antiguo del menú. El archivo AjustarFondo ayuda a que la portada cubra la vista sin deformarla.

11 SonidosJuego y la música

Dos reproductores con trabajos distintos

Un AudioClip es una grabación guardada. Un AudioSource es el reproductor que la hace sonar. SonidosJuego usa un reproductor para efectos y tiene una conexión aparte con la música del nivel. Esto permite cortar la música al ganar o perder.

Awake prepara el reproductor de efectos para que no empiece solo y para que se escuche sin depender de la distancia. También pide cargar las grabaciones con anticipación. Saltar y RecogerHoja reproducen su efecto si la partida sigue activa y hay una grabación conectada.

Los archivos que se usan

Momento	Archivo y comportamiento
Salto	SaltoBoing.wav efecto corto
Hoja	RecogerHoja.wav efecto de recogida
Derrota	DerrotaOofInmediato.wav sonido de muerte
Victoria	VictoriaPerrito.wav frase de celebración
Durante el nivel	MusicaArcade.wav se repite de fondo

Cómo se evita el retraso de la muerte

El archivo de derrota tenía silencio al principio. Se preparó una copia recortada para que el sonido empiece al recibir la orden. Morir pide el sonido antes de mostrar el panel. No se usó una espera programada para retrasarlo.

Perder y Ganar llaman al mismo bloque ReproducirResultado. Ese bloque recuerda que ya terminó, detiene la música, corta un efecto que estuviera sonando y reproduce el resultado una sola vez. Así no se acumulan varios memes por un mismo contacto.

La canción de fondo

MusicaDelNivel tiene un AudioSource que empieza al cargar el nivel, se repite y está a volumen bajo. Al reiniciar la escena vuelve a comenzar. Es Level 3 del paquete Retro Game Music Pack de Juhani Junkala, con licencia CC0. No es una canción de Metal Slug. La hoja tampoco usa una canción de Snoop Dogg en esta versión.

La licencia y el origen de la música están guardados en Documentacion/Licencia_musica_arcade.txt. Los efectos y memes tienen otros orígenes; no hay que presentar todos los recursos como si fueran hechos a mano o tuvieran la misma licencia.

12 El menú principal y sus botones

Cómo se armó la pantalla

Se creó MenuPrincipal aparte del nivel. Tiene una cámara, AdministradorMenu, un Canvas con imágenes y botones, y SistemaDeEventos para recibir los clics. El Canvas organiza las cosas que van encima de la vista del juego.

La portada actual se llama PortadaMarihuanero.jpg y fue proporcionada por ti. Se muestra completa en una parte de la composición y también se usa ampliada y oscurecida como fondo. El contenido del menú acompaña esa imagen. El lema visible dice PLATA O PLOMO y el botón principal dice JUGAR.

ControlMenuPrincipal

Awake devuelve el tiempo a velocidad normal y deja el cursor visible y libre. Esto evita entrar al menú con el tiempo detenido por un final anterior. La casilla escenaJuego guarda el nombre juegofuncional.

Jugar usa una marca llamada cargando para ignorar un segundo clic mientras empieza a abrir el nivel. Después pone el tiempo normal y carga juegofuncional. Para que funcione, el nombre debe coincidir con la escena y esta debe estar incluida en Build Profiles.

```
SceneManager.LoadScene(escenaJuego);
```

Salir hace dos cosas según dónde estemos: dentro del editor detiene Play; dentro del juego exportado pide cerrar la aplicación. El bloque que contiene UNITY_EDITOR separa esos dos casos.

Lo que todavía no hace

NIVELES está como botón de presentación y no tiene una acción conectada. No abre un selector de niveles. Es algo que se puede desarrollar después. El nombre interno BotonPlataOPlomo puede seguir apareciendo en Hierarchy, aunque el texto que ve el jugador ya diga JUGAR.

Cómo lo enseño

Abro MenuPrincipal y selecciono el botón JUGAR. En On Click muestro la conexión con AdministradorMenu y el método Jugar. Después doy Play, pulso el botón y enseño que abre la otra escena.

13 Los textos y las pantallas finales

El contador de hojas

El contador está dentro de Canvas, en un panel colocado arriba a la izquierda. Incluye el dibujo de una hoja y un texto de TextMeshPro. El programa cambia el contenido del texto; el color, tamaño y lugar se ajustan en el Inspector.

Los anclajes o Anchors indican a qué zona de la pantalla se pega el objeto. Por eso el contador usa la esquina superior izquierda. El Canvas Scaler ayuda a conservar la composición con distintas ventanas usando una referencia de 1920 por 1080.

Derrota y victoria

Los dos paneles cubren la pantalla y oscurecen el nivel para que el resultado se lea bien. Tienen una imagen del Patrón, un mensaje y un botón. El de derrota muestra ELEGISTE PLOMO y el de victoria felicita por completar el recorrido.

ControlNivel los esconde al empezar y muestra el correcto al terminar. Los botones llaman a ReiniciarNivel. Ocultar un panel con SetActive(false) no lo borra; solo deja de mostrarse hasta que el programa vuelva a activarlo.

Cómo se conecta una casilla

Primero selecciono el objeto que tiene el script, por ejemplo AdministradorNivel. Después busco el campo panelVictoria y arrastro PanelVictoria desde Hierarchy. Para un campo de texto arrastro el objeto que tiene el texto de TextMeshPro. Para un molde arrastro el prefab desde Project.

Reglas para trabajar sin perder cambios

Detengo Play antes de editar lo que quiero guardar. Guardo la escena con Ctrl más S y guardo los archivos de código en el editor. Los cambios hechos durante Play suelen volver a su estado anterior cuando detengo la prueba.

Si aparece New Text, selecciono el texto correcto y cambio Text Input. Si una llave del código falta, Console señala un error de escritura. Si el juego no responde como esperaba, primero miro Console y luego las conexiones del Inspector.

14 Cómo trabajan juntos los archivos

El viaje de una partida

1. Abro MenuPrincipal. ControlMenuPrincipal prepara el cursor y JUGAR abre juegofuncional.

2. ControlNivel prepara una partida limpia. Crea las hojas y pinchos desde sus moldes. La música del nivel empieza a repetirse.

3. ControlJugador lee las teclas y mueve el cuerpo. AnimarJugador escoge la pose. CamaraSigue acompaña el recorrido. AjustarFondo y AnimarSelva mantienen el ambiente visual.

4. Si recojo una hoja, ControlJugador reconoce la etiqueta y avisa a ControlNivel. El administrador suma una, cambia el texto y pide el sonido a SonidosJuego.

5. Si toco pinchos o caigo, ControlNivel decide derrota. Si toco la meta con diez hojas, decide victoria. SonidosJuego corta la música y reproduce el resultado. El panel aparece y el tiempo se detiene.

6. El botón vuelve a cargar la escena. Se ejecuta la preparación inicial y comienza otro intento.

Lo que ya se corrigió durante el desarrollo

Se ajustaron suelo y plataformas para que el dibujo coincida con el apoyo. Se usó un material sin fricción para evitar quedarse pegado a los lados. Se corrigió el seguimiento de cámara para reducir el tirón al saltar. Se recortó el silencio inicial del sonido de derrota. Los pinchos recibieron una imagen más agresiva conservando su zona de contacto.

Qué hace esta entrega de documentación

Los ocho scripts de Assets/Scripts recibieron comentarios por bloques con palabras sencillas. Se compararon las instrucciones antes y después de comentar y resultaron iguales. También quedó un respaldo de los originales en Documentacion/RespaldoScriptsAntesDeComentarios.

Qué puedo presentar como mejoras futuras

Un selector de niveles real, pausa, regreso al menú desde los resultados y más recorridos. No debo decir que NIVELES ya funciona ni que se probó una exportación completa si todavía no la he realizado. La base jugable y la presentación son distintas de esas mejoras pendientes.

15 Mi explicación para la exposición

Puedes practicar este texto en voz alta y cambiar las palabras para que suene a ti. Está pensado para unos tres o cuatro minutos con pausas cortas para señalar Unity.

Presento el objetivo

“Mi videojuego se llama Marihuanero. Es un juego de plataformas en 2D ambientado en una selva. La idea es recorrer el nivel, recoger diez hojas y llegar al Patrón. Si caigo en un hueco o toco los pinchos, pierdo y puedo volver a intentar”.

Muestro cómo lo organicé

“Usé Unity y separé el menú del nivel en dos escenas. En Project tengo carpetas para imágenes, sonidos, scripts y prefabs. Un prefab es un molde que permite crear varios objetos iguales. Por ejemplo, las hojas aparecen a partir de un solo molde”.

Explico el personaje y las reglas

“ControlJugador lee A, D, las flechas y espacio. El cuerpo tiene gravedad y una caja para los contactos. El salto solo se permite cuando encuentra suelo debajo de los pies. AnimarJugador cambia los dibujos según si está quieto, caminando o saltando”.

“ControlNivel guarda cuántas hojas llevo. Cuando toco una, suma una y actualiza el contador. Al tocar la meta revisa si ya tengo diez. También muestra victoria o derrota y permite reiniciar la partida”.

Explico la presentación

“La cámara sigue al personaje a los lados. El fondo se ajusta a la pantalla y la selva tiene una capa de niebla y luces. SonidosJuego reproduce los efectos. La música acompaña el nivel y se detiene cuando termina la partida para que se escuche el resultado”.

Cierro con lo aprendido

“Aprendí a separar lo que se ve de lo que se puede tocar, a conectar objetos con scripts y a revisar los mensajes de error. También entendí que el dibujo de un personaje no lo mueve por sí solo. Como siguiente mejora haría que el botón NIVELES permita escoger otros recorridos”.

Si preguntan por la ayuda recibida, responde con sinceridad: hubo asistencia para código y recursos visuales. Tu defensa debe demostrar que entiendes cómo están conectados, cómo se prueban y qué puedes cambiar.

16 Preguntas que me pueden hacer

Por qué usaste varios scripts

Porque cada uno tiene una tarea. Uno mueve, otro cuenta, otro anima y otro reproduce sonidos. Así puedo encontrar más fácil dónde cambiar algo.

Cómo sabe que recogiste una hoja

La caja del personaje entra en la zona de la hoja. El programa reconoce la etiqueta Hoja, suma una en el administrador y elimina esa copia.

Cómo evitas ganar antes de tiempo

La meta llama a IntentarGanar. Allí se compara la cuenta con diez. Si faltan hojas, solo muestra un aviso.

Cómo evitas saltar en el aire

Se revisa una caja pequeña debajo de los pies. Si no encuentra la capa Suelo, no se acepta el salto pendiente.

Qué diferencia hay entre Update y FixedUpdate

Update escucha al jugador mientras se dibuja el juego. FixedUpdate hace los cambios del cuerpo siguiendo los pasos de la física. LateUpdate coloca la cámara después de esos movimientos.

Qué diferencia hay entre una imagen y un collider

La imagen es lo que veo. El collider es la forma invisible que Unity usa para saber dónde se toca algo. Pueden tener tamaños distintos, por eso hay que ajustarlos.

Qué pasa cuando reinicias

El tiempo vuelve a uno y se carga otra vez la escena. Start pone la cuenta en cero, esconde los paneles y vuelve a crear los objetos.

Por qué no se ven plataformas al abrir el menú

Porque Hierarchy solo muestra la escena abierta. Las plataformas siguen en juegofuncional. Abrir MenuPrincipal no las elimina.

Para qué sirven los comentarios nuevos

Explican qué hace cada bloque con palabras sencillas. Las líneas que empiezan con dos barras no se ejecutan. Ayudan a estudiar sin cambiar las reglas del juego.

17 Repaso final antes de presentar

Palabras que sí debo entender

Palabra	Cómo la explico fácil
Variable	Un espacio donde guardo un dato como la cuenta de hojas
Método	Un grupo de instrucciones con un nombre para pedir una tarea
Condición if	Una pregunta que decide si hacemos algo
Bucle foreach	Repetir una tarea para cada elemento de una lista
public	Permite que otras partes usen ese dato y puede mostrar una casilla
private	Dato que se maneja dentro de ese script
GameObject	Un objeto del juego que puede tener varias piezas
Componente	Una de esas piezas como cámara caja o sonido
Trigger	Una zona que avisa que entramos sin ser una pared
Referencia	Una conexión hacia el objeto que otro bloque necesita

Ensayo de cinco minutos

Abro MenuPrincipal y explico JUGAR. Camino y salto. Recojo una hoja y señalo el contador. Toco un peligro para mostrar derrota y el botón de reintento. Después enseño en el código la revisión de diez hojas para explicar la victoria sin depender de completar todo el nivel durante la exposición.

Antes de comenzar, compruebo que el audio se escucha a un volumen cómodo, que no hay errores rojos y que tengo abiertas las carpetas Scripts y Scenes. Guardo y salgo de Play para comenzar desde el menú.

Si se me olvida una respuesta

Busco la tarea por su nombre: movimiento en ControlJugador, reglas en ControlNivel, dibujos en AnimarJugador, cámara en CamaraSigue, fondo en AjustarFondo y AnimarSelva, audio en SonidosJuego y menú en ControlMenuPrincipal. Explico el bloque con calma y señalo el objeto conectado.

Lo más valioso no es recitar el código: es poder mostrar qué dato entra, qué decisión toma el programa y qué cambia en la pantalla.

18 Código de ControlJugador

Archivo ControlJugador.cs dentro de Assets/Scripts. Está en Jugador. Estos fragmentos son partes de FixedUpdate y OnTriggerEnter2D.

Preparar el movimiento y aceptar el salto

```
Vector2 velocidad = new Vector2(
    direccion * rapidez,
    cuerpo.linearVelocity.y
);

if (saltoPendiente && enSuelo)
{
    velocidad.y = fuerzaSalto;
    nivel.sonidos?.Saltar();
}

cuerpo.linearVelocity = velocidad;

saltoPendiente = false;
```

Primero preparo dos números: el avance a los lados y la velocidad vertical que ya tenía. La pregunta if exige que se haya pedido el salto y que haya piso debajo. El signo && significa que deben cumplirse las dos cosas. Entonces cambio la velocidad vertical, pido el sonido y entrego la velocidad al cuerpo. Al final borro la petición de salto.

Reconocer una hoja

```
if (otro.CompareTag("Hoja"))
{
    otro.enabled = false;
    nivel.RecogerHoja();
    Destroy(otro.gameObject);
}
```

Reconozco la etiqueta Hoja. Apago su caja para no contarla dos veces, aviso al administrador y borro esa copia. Destroy no elimina el molde de Project.

Así lo explico al profesor

“Este bloque mueve el cuerpo y solo le permite saltar cuando está apoyado. El otro bloque reconoce una hoja y le avisa al encargado del nivel para sumarla. El personaje reconoce el contacto; el administrador guarda la cuenta”.

19 Código de ControlNivel

Archivo ControlNivel.cs dentro de Assets/Scripts. Está en AdministradorNivel. Aquí se cuentan las hojas y se decide el resultado.

Sumar una hoja y cambiar el texto

```
public void RecogerHoja()
{
    if (PartidaTerminada) return;
    hojasRecogidas++;
    sonidos?.RecogerHoja();
    ActualizarContador();
}

void ActualizarContador()
{
    textoHojas.text = "Hojas: " + hojasRecogidas + " / " + totalHojas;
}
```

Si ya terminó, return sale del método. Si seguimos jugando, ++ suma una hoja. Después pide el sonido y actualiza el texto. Los signos + unen palabras y números para escribir la cuenta. El signo ?. pide ejecutar el sonido solo si esa conexión existe.

Lo que ocurre al perder

```
public void Morir()
{
    if (PartidaTerminada) return;
    derrotaMostrada = true;
    sonidos?.Perder();
    panelDerrota.SetActive(true);
    Time.timeScale = 0f;
}
```

La primera pregunta evita repetir la derrota. Después guardo que perdimos, pido el sonido, enciendo el panel y dejo quieto el juego. La victoria sigue una idea parecida, pero antes compara hojasRecogidas con totalHojas dentro de IntentarGanar.

Así lo explico al profesor

“ControlNivel es el encargado de las reglas. No lee el teclado: recibe avisos. Al recoger una hoja suma una y cambia el contador. Cuando perdemos, muestra el resultado una sola vez y pausa el movimiento”.

20 Código de AnimarJugador

Archivo AnimarJugador.cs dentro de Assets/Scripts. Está en Jugador y controla el dibujo de ImagenJugador. Estos bloques están en MostrarMovimiento y MostrarPoseSalto.

Elegir entre caminar y estar quieto

```
int siguiente = Mathf.Abs(velocidad.x) > .05f ? Caminar : Quieto;
animador.speed = 1f;
if (estadoActual != siguiente)
{
    animador.Play(siguiente, 0, 0f);
    estadoActual = siguiente;
}
```

Mathf.Abs mira cuánto se mueve hacia un lado sin importar si es izquierda o derecha. Si supera 0,05 elige Caminar; si no, Quieto. La parte con ? y : significa “si pasa esto elige lo primero y si no elige lo segundo”. Luego evita comenzar de nuevo una animación que ya estaba sonando.

Mostrar una pose del salto

```
void MostrarPoseSalto(float fase)
{
    animador.speed = 0f;
    animador.Play(Saltar, 0, fase);
    estadoActual = Saltar;
}
```

Pongo la velocidad del animador en cero para dejar fija una pose del salto. Play elige Saltar y la parte indicada por fase. Esto no congela el cuerpo: solo el dibujo. El personaje sigue subiendo o cayendo con su física.

Así lo explico al profesor

“Primero miro cómo se está moviendo el cuerpo y luego elijo la imagen. Si camina cambio a Caminar; si se detiene uso Quieto. En el salto elijo una pose que combine con su subida o caída. Este script no le da fuerza al personaje”.

21 Código de CamaraSigue

Archivo CamaraSigue.cs dentro de Assets/Scripts. Está en Main Camera. Los fragmentos pertenecen a LateUpdate.

Acompañar el avance del personaje

```
float avance = jugador.position.x - ultimaXJugador;
ultimaXJugador = jugador.position.x;
transform.position += new Vector3(avance, 0f, 0f);
```

Resto la posición de antes a la de ahora para saber cuánto avanzó. Guardo la posición nueva para la próxima revisión. Después muevo la cámara esa misma distancia a los lados. Como los otros dos valores son cero, este bloque no cambia ni su altura ni su profundidad.

Corregir el encuadre con suavidad

```
Vector3 destino = new Vector3(
    jugador.position.x + 2f,
    transform.position.y,
    transform.position.z
);

transform.position = Vector3.SmoothDamp(
    transform.position,
    destino,
    ref velocidad,
    suavizado
);
```

El destino queda dos unidades delante del personaje. Conservo la Y y la Z de la cámara. SmoothDamp acerca la cámara a ese destino de forma suave cuando hay una diferencia. suavizado indica cómo se hace esa corrección; velocidad guarda un dato que necesita ese cálculo entre revisiones.

Así lo explico al profesor

“La cámara acompaña lo que avanzó el jugador y deja ver un poco de camino por delante. No sube con cada salto porque conserva su altura. Eso ayuda a que el personaje no parezca retroceder por culpa del encuadre”.

Para acercar la vista se cambió Size en el componente Camera, no en este bloque. El valor guardado es 4,2. Es un ajuste del Inspector que trabaja junto al seguimiento.

22 Código de AjustarFondo

Archivo AjustarFondo.cs dentro de Assets/Scripts. Se usa en los fondos con Sprite Renderer. Este fragmento está dentro de AjustarTamano.

Calcular cuánto debe crecer la imagen

```
float altoPantalla = camara.orthographicSize * 2f;
float anchoPantalla = altoPantalla * camara.aspect;
float escala = Mathf.Max(
    anchoPantalla / tamanoImagen.x,
    altoPantalla / tamanoImagen.y
) * 1.002f;
```

orthographicSize es la mitad de la altura que ve la cámara, por eso lo multiplico por dos. aspect dice la relación entre ancho y alto. Las divisiones comparan el espacio visible con el tamaño del dibujo. Mathf.Max elige el aumento mayor para que la imagen cubra toda la pantalla. El 1,002 deja un margen muy pequeño en los bordes.

Aplicar el cambio solo cuando hace falta

```
if ((transform.localScale - nuevoTamano).sqrMagnitude > .00000001f)
    transform.localScale = nuevoTamano;
```

nuevoTamano se preparó antes teniendo en cuenta el tamaño del objeto padre. Esta pregunta compara ese tamaño con el que ya tiene la imagen. Si son prácticamente iguales no vuelve a cambiarlo. No es necesario memorizar el número pequeño: sirve como margen para decidir si la diferencia importa.

Así lo explico al profesor

“Este bloque calcula qué tan grande debe ser el fondo para llenar la vista sin deformar el dibujo. Usa el mismo aumento para ancho y alto. Puede recortar un borde de la imagen, pero evita dejar un espacio vacío alrededor”.

Si quiero demostrarlo, cambio la forma de la ventana Game y señalo que el fondo sigue cubriendo la vista. Esto modifica la apariencia; no cambia las cajas del piso ni el movimiento del personaje.

23 Código de AnimarSelva

Archivo AnimarSelva.cs dentro de Assets/Scripts. Está en la capa AtmosferaSelva del nivel. Da valores al material que pinta la niebla y las luces.

Hacer que el ciclo vuelva a empezar

```
void LateUpdate()
{
    if (Application.isPlaying)
        segundos = Mathf.Repeat(segundos + Time.deltaTime, Mathf.Max(1f, duracionCiclo));
    ActualizarCapa();
}
```

Application.isPlaying pregunta si estamos probando el juego. Time.deltaTime dice cuánto tiempo pasó desde la última imagen. Lo sumo al reloj. Mathf.Repeat hace que el reloj vuelva al principio cuando llega a la duración del ciclo. Luego actualizo la apariencia. Al detenerse el tiempo del juego, este reloj deja de avanzar.

Enviar los valores al dibujo

```
propiedades.SetFloat(Fase, segundos / Mathf.Max(1f, duracionCiclo));
propiedades.SetFloat(Aspecto, camara.aspect);
propiedades.SetFloat(Neblina, cantidadNeblina);
propiedades.SetFloat(Rayos, intensidadRayos);
propiedades.SetFloat(Particulas, brilloParticulas);
```

Aquí envío por qué parte del ciclo vamos, la forma de la pantalla y cuánto se ven niebla, rayos y puntitos. Estas líneas preparan los valores. Más abajo se aplican a la capa con SetPropertyBlock. El shader los recibe y decide los colores transparentes que se dibujan.

Así lo explico al profesor

“No usé un GIF. Tengo un reloj que se repite y le dice al material cómo variar la niebla, la luz y los puntitos. La imagen de selva es el fondo y esta capa le da movimiento. Las cajas del suelo siguen siendo las mismas”.

La duración inicial del ciclo es diez segundos. También puedo cambiar la cantidad de niebla desde una casilla, sin dibujar otro fondo ni cambiar el recorrido.

24 Código de SonidosJuego

Archivo SonidosJuego.cs dentro de Assets/Scripts. Está en AdministradorNivel. Usa un reproductor para los efectos y una conexión a la música.

Pedir un efecto al recoger una hoja

```
public void RecogerHoja()
{
    if (!partidaTerminada && hoja != null)
        fuente.PlayOneShot(hoja, volumenHoja);
}
```

El signo ! significa “no”. La pregunta pide que la partida no haya terminado y que exista el audio de hoja. PlayOneShot lo reproduce una vez con el volumen elegido. La grabación hoja y el reproductor fuente son cosas distintas: uno es el archivo y el otro lo hace sonar.

Escuchar el resultado una sola vez

```
private void ReproducirResultado(AudioClip sonido)
{
    if (partidaTerminada) return;
    partidaTerminada = true;
    if (musicaFondo != null) musicaFondo.Stop();
    fuente.Stop();
    if (sonido != null) fuente.PlayOneShot(sonido, volumenResultado);
}
```

Primero evito repetir el final. Después marco que terminó, detengo la música si existe y corto los efectos anteriores. Finalmente reproduzco la grabación recibida. Perder le entrega derrota y Ganar le entrega victoria. El mismo bloque sirve para los dos resultados.

Así lo explico al profesor

“Cuando recojo una hoja se pide un efecto corto. Al ganar o perder uso otro bloque que corta la música y deja escuchar el resultado una sola vez. La muerte suena pronto porque el archivo de audio tiene recortado el silencio inicial”.

No hay una espera escondida en este método. El orden de las instrucciones permite pedir el sonido antes de mostrar el panel y detener el movimiento.

25 Código de ControlMenuPrincipal

Archivo ControlMenuPrincipal.cs dentro de Assets/Scripts. Está en AdministradorMenu dentro de MenuPrincipal. Los botones llaman a estos métodos.

Abrir el nivel al pulsar JUGAR

```
public void Jugar()
{
    if (cargando) return;
    cargando = true;
    Time.timeScale = 1f;
    SceneManager.LoadScene(escenaJuego);
}
```

Si ya se pidió abrir el nivel, return evita repetir la orden. Si todavía no, guardo cargando como verdadero, pongo el tiempo normal y cargo la escena. escenaJuego es una casilla que contiene juegofuncional. El nombre debe coincidir y la escena debe estar en la lista de Build Profiles.

Salir según dónde se está ejecutando

```
public void Salir()
    {
#if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
#else
        Application.Quit();
#endif
    }
```

Dentro de Unity se usa la parte UNITY_EDITOR para detener Play. En el juego exportado se usa la otra parte para cerrar la aplicación. Las líneas que empiezan con # hacen que Unity elija el bloque correspondiente al preparar el programa.

Así lo explico al profesor

“El menú no contiene las plataformas. Solo presenta el juego y abre la escena del nivel. Jugar carga juegofuncional y Salir cierra la prueba o el juego según dónde lo esté usando. El botón NIVELES está reservado para una mejora y todavía no tiene acción”.

Cómo estudio estos fragmentos

Leo primero el nombre del bloque. Después digo qué dato recibe, qué pregunta hace y qué cambia en el juego. Si me piden más detalle, abro el archivo en Assets/Scripts y uso los comentarios que ya están escritos encima de cada parte.